DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-3

ASSIGNMENT 3 – CO3

CO3 Coverage: Context-Free Grammars for English syntax; Context-free rules and trees; Sentence-level constructions; Agreement; Subcategorization; Parsing (Top down, Earley); Feature structures; Probabilistic Context-Free Grammars

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: ___04-11-2026_____

Assignment Title: Design, Parse and Probabilistically Rank English Queries for a University Helpdesk Using Context-Free Grammars, Feature Structures and Earley Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Syntactic Analysis for a University Helpdesk Assistant A university is building a text-based helpdesk assistant that answers student and staff queries about timetables, examinations, hostels, fees, library services and placements. Before any meaning can be extracted, every query must be assigned a reliable syntactic structure. The NLP team has decided to build a grammar-based parser for a controlled but realistic fragment of English, to resolve ambiguity with a probabilistic model, and to compare parsing algorithms on accuracy and cost.

You are the language-engineering team. You must investigate how English syntax can be captured by context-free rules, how agreement and subcategorization constrain those rules, how parsing algorithms search for trees, and how probabilities select the best tree when a query is ambiguous.

Planning data and constraints

- Grammar fragment: at least 40 phrase-structure rules and a lexicon of at least 60 words covering the helpdesk domain.
- Sample query set (build and document your own): at least 30 sentences, including declaratives, imperatives, yes/no questions, wh-questions and at least 5 structurally ambiguous sentences (e.g., “Show the students with the late fee notice”, “Book the room near the library for the seminar”).

• Required phenomena: subject–verb agreement, determiner–noun number agreement, pronoun case, and at least five verb subcategorization frames. • The grammar must be written in Chomsky-style CFG notation; for the Earley and probabilistic tasks the grammar may be converted to a binarised or CNF equivalent form where needed, and the conversion must be shown. • Treebank probabilities: a small hand-annotated corpus of at least 20 parsed sentences may be used to estimate rule probabilities (maximum-likelihood estimates with smoothing explained).

- Any manual effort (grammar writing, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The helpdesk is to be extended to accept noisy, conversational and multi-clause student messages (e.g., "Can you tell me when the exams that were postponed will be held?"). Re-evaluate your original grammar and parser and decide whether to retain, modify or redesign them.

TASK 1 – Design a Context-Free Grammar for English Syntax 5

Marks Guiding Enquiry

How can the core structure of English be captured by a finite set of context-free rules, and what design decisions make the grammar both expressive and manageable?

Investigate and interpret:

• Define the formal CFG **G = (N, Σ, R, S)** for your helpdesk fragment and state clearly the non-terminals, terminals (lexicon), rules and start symbol. • Write rules for the major constituents: sentence (S), noun phrase (NP), verb phrase (VP), prepositional phrase (PP), adjective phrase, and coordination. Include recursive rules (e.g., NP → NP PP) and explain how they permit unbounded structure.

- Separate the grammar proper from the lexicon; list the parts of speech used and justify the granularity you selected.
- Explain the notions of constituency, head, and phrasal category with tests (substitution, movement, coordination) applied to at least three of your sentences.
- Show that your grammar is context-free by construction and discuss one English phenomenon that strains the context-free assumption.

Evidence Expected

A complete, well-organised CFG with lexicon, justified categories and worked constituency tests.

1.1 Definition of CFG

A Context-Free Grammar (CFG) is represented as:

\[G=(N,\Sigma,R,S) \]

Where:

- \(N\): non-terminal symbols such as S, NP, VP, PP.
- \(\Sigma\): terminal symbols, or words in the lexicon.
- \(R\): production rules.
- \(S\): start symbol.

A CFG rule has the form:

\[A \rightarrow \beta \]

Here, \(A\) is one non-terminal and \(\beta\) is a sequence of terminals and/or non-terminals.

1.2 Phrase-structure rules

A sample CFG for a university helpdesk:

S → NP VP

S → VP

S → AUX NP VP

S → WH NP VP

S → WH AUX NP VP

NP → DET N

NP → DET ADJ N

NP → PN

NP → PRON

NP → NP PP

NP → NP CONJ NP

VP → V

VP → V NP

VP → V NP NP

VP → V PP

VP → V S

VP → AUX VP

VP → VP PP

VP → VP CONJ VP

PP → P NP

AP → ADJ

AP → ADJ PP

CP → COMP S

REL → RELPRO VP

N → N REL

Recursive rules such as NP → NP PP allow a noun phrase to contain additional prepositional phrases. This permits structures of increasing length, such as:

the room near the library beside the main gate

1.3 Lexicon

Part of speech	**Example words**
Determiner (DET)	the, a, this, those
Noun (N)	student, fee, exam, hostel, library, room, timetable
Proper noun (PN)	Ravi, Anna
Pronoun (PRON)	I, we, he, she, them
Verb (V)	submit, pay, show, book, crash, apply, tell

Auxiliary (AUX)	do, does, can, will, is, are
Preposition (P)	in, on, near, for, with, to
Adjective (ADJ)	late, final, available, new
Wh-word (WH)	who, what, where, when, which
Conjunction (CONJ)	and, or, but
Complementizer (COMP)	that, whether
Relative pronoun (RELPRO)	who, which, that

1.4 Constituency, head, and phrasal category

A constituent is a group of words that functions as a unit. The head is the central word that determines the phrase's category.

Example: the late fee notice

- Head: notice
- Phrase category: NP
- Determiner: the
- Adjectives: late, fee (in this compound-like phrase)

Constituency tests:

1. Substitution: "The library is open." → "It is open."
   The library behaves as an NP.
2. Movement: "The room near the library is available." → "Near the library, the room is available."
   near the library behaves as a PP.
3. Coordination: "Students can [pay the fee] and [submit the form]."
   The two VPs can be coordinated.

1.5 Why the grammar is context-free

Each rule rewrites one non-terminal without depending on surrounding symbols. Therefore, the grammar is context-free by construction.

A phenomenon that strains CFGs is cross-serial dependency, where multiple dependencies overlap in a way that is difficult to represent with a simple CFG. For a controlled helpdesk grammar, CFG is still useful because it handles common English sentence structures clearly.

TASK 2 – Context-Free Rules, Derivations and Parse Trees 3 Marks

Guiding Enquiry

How are sentences generated from the grammar, and how are trees used to represent and compare syntactic analyses?

Investigate and interpret:

- For five sentences from your query set, give a leftmost derivation and a rightmost derivation, and draw the corresponding parse tree.
- Represent each tree in both bracketed notation and tree diagram form, and compute its height, number of nodes and number of rule applications. • Demonstrate ambiguity: for at least two sentences, give two distinct parse trees and describe the difference (e.g., PP-attachment, coordination scope). • Explain the relationship between derivations and trees, and why many derivations may correspond to one tree.
- State a procedure to test whether a given string belongs to L(G) and discuss its limits.

Evidence Expected

Correct derivations, accurately drawn trees, and a clear analysis of structural ambiguity.

2.1 Example sentence

Sentence:

"The student pays the fee."

Grammar rules:

S → NP VP

NP → DET N

VP → V NP

Lexicon:

DET → the

N → student | fee

V → pays

2.2 Leftmost derivation

S

⇒ NP VP

⇒ DET N VP

⇒ the N VP

⇒ the student VP

⇒ the student V NP

⇒ the student pays NP

⇒ the student pays DET N

⇒ the student pays the N

⇒ the student pays the fee

2.3 Rightmost derivation

S

⇒ NP VP

⇒ NP V NP

⇒ NP V DET N

⇒ NP V DET fee

⇒ NP V the fee

⇒ NP pays the fee

⇒ DET N pays the fee

⇒ DET student pays the fee

⇒ the student pays the fee

2.4 Bracketed tree

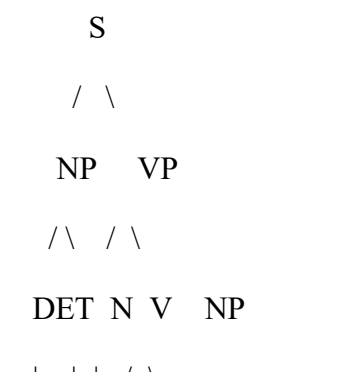
[S

[NP [DET The] [N student]]

[VP [V pays]

[NP [DET the] [N fee]]]]

2.5 Tree diagram



- Height: 4 edges from root to deepest leaf.
- Number of nodes: 10, including word nodes.
- Rule applications: 5, counting the phrase rules and lexical expansions.

2.6 Ambiguity

Sentence: “Show the students with the late fee notice.”

Possible readings:

1. NP attachment: The students have the late fee notice.
2. VP attachment: Show the students using the late fee notice.

The two trees differ in whether with the late fee notice attaches to NP or VP.

A string belongs to \(L(G)\) if the grammar can derive it from the start symbol. A parser can test this by searching for a derivation. The limitation is that a CFG may accept a structurally valid sentence that is semantically incorrect.

TASK 3 – Sentence-Level Constructions 4 Marks

Guiding Enquiry

Which sentence types must the helpdesk understand, and how does the grammar represent each of them?

Investigate and interpret:

• Provide rules for the four main sentence-level constructions: declaratives, imperatives, yes/no questions and wh-questions (subject and non-subject). • Handle auxiliary inversion, wh-movement (represented through a gap or an additional rule), and subordinate and relative clauses (e.g., “the exams that were postponed”).

• Show parse trees for at least one sentence of each construction type and note the structural differences from the declarative.

• Discuss how coordination (“fees and hostel charges”) and long-distance dependencies are treated and where the plain CFG becomes unwieldy. • Analyse the effect of each new construction on grammar size and ambiguity.

Evidence Expected

Rules and trees for each sentence type, with an analysis of how the added constructions affect the grammar.

3.1 Main sentence types

DECLARATIVE → S → NP VP

IMPERATIVE → S → VP

YES/NO Q → S → AUX NP VP

WH Q → S → WH AUX NP VP

Examples:

Type	**Example**
Declarative	The exam starts today.
Imperative	Submit the form.
Yes/no question	Does the exam start today?
Wh-question	When does the exam start?

3.2 Auxiliary inversion

Declarative:

The exam will start today.

Question:

Will the exam start today?

The auxiliary will moves before the subject NP.

[S

[AUX Will]

[NP the exam]

[VP start today]]

3.3 Wh-movement

Example: When will the exam start?

The wh-word when appears at the beginning, while its semantic position is associated with the VP.

[S

[WH When]

[AUX will]

[NP the exam]

[VP start __]]

The blank represents a gap.

3.4 Subordinate and relative clauses

Example:

The exams that were postponed will be held tomorrow.

The relative clause that were postponed modifies exams.

[S

[NP

[DET The]

[N exams]

[REL that were postponed]]

[VP will be held tomorrow]]

3.5 Coordination and long-distance dependencies

Example: Fees and hostel charges are due.

NP → NP CONJ NP

Coordination combines two phrases of the same category. Long-distance dependencies, such as What did the student say that the teacher announced?, require the grammar to connect a wh-word with a gap inside an embedded clause.

A plain CFG can represent these constructions using additional rules, but the grammar grows and may generate many unwanted combinations. Feature structures can track agreement and gap information more economically.

TASK 4 – Agreement 3 Marks

Guiding Enquiry

How can a grammar prevent sentences such as "*The students submits the form" without producing an enormous set of rules?

Investigate and interpret:

- Identify the agreement phenomena in your fragment: subject–verb (number, person), determiner–noun, and pronoun case.
- First, enforce agreement by multiplying non-terminals (e.g., S → NP-sg VP-sg | NP-pl VP-pl). Count how many rules this requires for your grammar and explain the growth.
- Show how the rule explosion worsens as person, number, case and gender are combined, and compute the number of rules needed in the worst case. • Propose a more economical alternative (feature-based rules) and show informally how it prevents ungrammatical strings; the formal treatment follows in Task 8.
- Test the original and agreement-aware grammars on at least six grammatical and six ungrammatical sentences and tabulate the results.

Evidence Expected

A quantified comparison of the baseline and agreement-aware grammars with test results.

4.1 Agreement phenomena

Agreement includes:

- Subject–verb agreement: The student submits; the students submit.

- Determiner–noun agreement: This student; these students.
- Pronoun case: She submitted the form; the staff spoke to her.

4.2 Agreement using multiplied categories

A basic grammar may use:

S → NP-SG VP-SG

S → NP-PL VP-PL

It also needs separate noun phrases, verb phrases, determiners, and verb forms for each number and person.

If there are 3 persons × 2 numbers, there are 6 person-number combinations. If case has 3 values and gender has 3 values, the worst-case feature combinations are:

$$ 3 \times 2 \times 3 \times 3 = 54 $$

If each combination needs a separate rule, grammar size increases rapidly.

4.3 Feature-based alternative

Instead of creating separate categories, use features:

NP: [NUM=SG, PER=3]

VP: [NUM=SG, PER=3]

The S rule requires the subject and verb phrase to share the same agreement values.

Thus:

- The student submits the form. — accepted
- The students submits the form. — rejected

4.4 Test examples

Sentence	**Agreement-aware grammar**
The student submits the form.	Accept
The students submit the form.	Accept
These students pay the fee.	Accept
This student pays the fee.	Accept

She submits the form.	Accept
They submit the form.	Accept
*The students submits the form.	Reject
*These student pay the fee.	Reject
*This students pays the fee.	Reject
*Her submit the form.	Reject
*Me submits the form.	Reject
*They pays the fee.	Reject

TASK 5 – Subcategorization 4 Marks

Guiding Enquiry

How does the grammar know which complements a verb may take, and how can this knowledge be encoded?

Investigate and interpret:

- Define subcategorization (valence) and list at least five frames for helpdesk verbs, e.g., intransitive (“The portal crashed”), transitive (“Submit the form”), ditransitive (“Send me the receipt”), sentential complement (“Tell me that the fee is paid”), and PP complement (“Apply for a hostel room”).
- Encode the frames first with distinct verb categories (V-intr, V-trans, V-ditrans, V-comp, …) and then explain the drawbacks of this approach.
- Distinguish complements from adjuncts using tests, with examples.
- Demonstrate how the grammar rejects over- and under-complemented sentences (“*Submit”, “*The portal crashed the form”) and show this in the parse results.
- Discuss verbs with multiple frames and how this changes ambiguity and parsing cost.

Evidence Expected

A subcategorization lexicon, grammar rules enforcing it, and tested accept/reject examples.

5.1 Definition

Subcategorization, also called valence, specifies the complements a verb requires.

Frame	Example
Intransitive: V	The portal crashed.
Transitive: V NP	Submit the form.
Ditransitive: V NP NP	Send me the receipt.
Sentential: V S	Tell me that the fee is paid.
PP complement: V PP	Apply for a hostel room.

5.2 Distinct verb categories

VP → V-INTR

VP → V-TRANS NP

VP → V-DITRANS NP NP

VP → V-COMP S

VP → V-PP PP

This prevents a verb from taking the wrong complement. However, it increases the number of verb categories and may be difficult when one verb has multiple frames.

5.3 Complement vs adjunct

A complement is selected by the verb. An adjunct adds optional information.

- Complement: apply for a hostel room
- Adjunct: submit the form tomorrow

The verb submit requires an object in this use, while tomorrow adds optional time information.

5.4 Rejecting incorrect sentences

Sentence	Result	Reason
The portal crashed.	Accept	Intransitive frame
Submit the form.	Accept	Transitive frame
*Submit.	Reject in this intended frame	Missing required object

*The portal crashed the form.	Reject	crash is intransitive in this meaning
Send me the receipt.	Accept	Ditransitive frame

Some verbs allow more than one frame. For example, book can be transitive (Book a room) or take an additional PP (Book a room for the seminar). Multiple frames increase ambiguity and may increase parser search.

TASK 6 – Top-Down Parsing 4 Marks

Guiding Enquiry

How does a top-down parser search for a tree rooted in S, and what are the strengths and weaknesses of this strategy?

Investigate and interpret:

- Describe the top-down (recursive-descent, goal-driven) algorithm in pseudocode, including how it expands non-terminals, matches words and backtracks.
- Trace the parser step by step on one unambiguous and one ambiguous sentence, showing the stack/agenda, the rule chosen and each backtracking point.
- Explain the problems of left recursion (e.g., NP → NP PP) and repeated re parsing of the same subtrees, and show these problems on a concrete example.
- Show how left-recursion elimination or a bottom-up filter (left-corner) addresses these issues and discuss the effect on the resulting trees.
- Analyse the worst-case time behaviour and the number of redundant sub parses for a sentence with k prepositional phrases.

Evidence Expected

Pseudocode, detailed traces and a correct analysis of the weaknesses of top-down parsing.

6.1 Algorithm

A top-down parser begins with the start symbol S and tries to derive the input sentence.

```
TOP-DOWN-PARSE(S, words):
   expand S using grammar rules
   if a terminal matches the next input word:
      consume the word
```

if a non-terminal is reached:

expand it using its rules

if a choice fails:

backtrack to the previous choice

accept if all words are consumed

and the start symbol is completed

6.2 Example trace

Sentence: The student pays the fee.

S

→ NP VP

→ DET N VP

→ The student VP

→ The student V NP

→ The student pays DET N

→ The student pays the fee

The parser succeeds because each predicted category matches the next input word.

6.3 Ambiguous attachment

Sentence: Show the students with the notice.

The parser first tries one possible PP attachment. If that structure fails or another parse is requested, it backtracks and tries attaching the PP to the other phrase.

6.4 Left recursion problem

Rule:

NP → NP PP

A naïve recursive-descent parser may repeatedly call NP before consuming any input. This can cause infinite recursion.

A common transformation is:

NP → NP_BASE NP_TAIL

NP_TAIL → PP NP_TAIL | ε

This removes direct left recursion while retaining the intended repeated-PP structure.

6.5 Complexity

A naïve top-down parser may repeatedly explore the same subtrees. In highly ambiguous grammars, its worst-case behaviour can be exponential. With \(k\) PPs, different attachment choices can create up to roughly \(2^k\) possibilities in a simplified binary-attachment model.

TASK 7 – Earley Parsing 5 Marks

Guiding Enquiry

How does the Earley algorithm use dynamic programming to avoid the failures of simple top-down parsing, and what does its chart contain?

Investigate and interpret:

- Define an Earley state (dotted rule with start position) and explain the three operations: PREDICTOR, SCANNER and COMPLETER.
- Hand-simulate the algorithm on a helpdesk sentence of at least six words; show every chart entry (S[0]…S[n]) with its state, origin and the operation that created it.
- Recover the parse tree(s) from the chart by adding back-pointers and show how multiple trees are packed for an ambiguous sentence.
- Show how the algorithm handles a left-recursive rule and an ε-free ambiguous attachment without looping.
- Derive the complexity ($O(n^3)$ time, $O(n^2)$ space for the chart) and compare it with the top-down parser of Task 6 on the same sentence in terms of number of states generated.

Evidence Expected

A fully traced Earley chart, back-pointer tree recovery, and a justified complexity comparison.

7.1 Earley state

An Earley state is represented as:

\[[A \rightarrow \alpha \bullet \beta,\ i] \]

- \(A \rightarrow \alpha\beta\): grammar rule
- Dot: how much of the rule has been recognized
- \(i\): input position where the state began

7.2 Three operations

1. Predictor: If the dot is before a non-terminal, add its possible rules.
2. Scanner: If the dot is before a terminal matching the next word, advance the dot.
3. Completer: When a rule is complete, advance states that were waiting for that non-terminal.

7.3 Example

Sentence: The student pays the fee.

Use:

S' → S

S → NP VP

NP → DET N

VP → V NP

DET → the

N → student | fee

V → pays

Initial chart:

S[0]:

[S' → • S, 0]

[S → • NP VP, 0]

[NP → • DET N, 0]

[DET → • the, 0]

After scanning the, student, pays, the, and fee, the completed chart contains:

[S' → S •, 0]

This indicates that the full sentence has been recognized.

7.4 Left recursion and ambiguity

Earley parsing stores states in a chart. If the same state is produced again, it is not repeatedly processed. This helps prevent infinite loops from left-recursive rules.

For PP attachment, the chart can retain more than one completed analysis and pack them using shared chart states and back-pointers.

7.5 Complexity

For a general CFG:

- Worst-case time: \(O(n^3)\)
- Chart space: \(O(n^2)\), with grammar-dependent state factors

Earley parsing is generally more robust than naïve top-down parsing because it reuses completed and predicted states. Exact state counts must be obtained from the implemented parser.

TASK 8 – Feature Structures and Unification 4 Marks

Guiding Enquiry

How can attribute–value structures and unification replace multiplied categories to handle agreement and subcategorization elegantly?

Investigate and interpret:

- Define feature structures (attribute–value matrices), feature paths, reentrancy (structure sharing) and represent at least five lexical entries with features such as NUM, PER, CASE and SUBCAT.
- Define unification and subsumption, and work through at least three unifications, including one that fails.
- Augment at least six grammar rules with feature constraints (e.g., S → NP VP with ⟨ NP AGREEMENT⟩ = ⟨ VP AGREEMENT⟩) and show the unified feature structure for one complete parse.
- Redo Task 4 (agreement) and Task 5 (subcategorization) using features and compare the grammar size with the earlier non-feature version.
- Describe how unification can be integrated with the Earley parser (unification during the COMPLETER step) and what is lost in complexity.

Evidence Expected

Correct feature structures and unifications, a feature-based grammar fragment, and a comparison with the non-feature grammar.

8.1 Feature structures

A feature structure represents grammatical information as attribute-value pairs.

Example:

[CAT = NP

NUM = SG

PER = 3

CASE = NOM]

- Feature path: A path to a feature, such as NP.NUM.
- Reentrancy: Two paths share the same feature value.
- Unification: Combining compatible feature structures.
- Subsumption: A more general feature structure is compatible with a more specific one.

8.2 Lexical entries

Word	**Feature structure**
student	[CAT=N, NUM=SG]
students	[CAT=N, NUM=PL]
submits	[CAT=V, NUM=SG, PER=3, SUBCAT=NP]
submit	[CAT=V, NUM=PL, SUBCAT=NP]
her	[CAT=PRON, CASE=ACC]

8.3 Unification examples

Successful:

[NUM=SG] ⊔ [NUM=SG] = [NUM=SG]

Successful:

[CAT=NP, NUM=PL] ⊔ [NUM=PL] = [CAT=NP, NUM=PL]

Failure:

[NUM=SG] ⊔ [NUM=PL] = failure

The last unification fails because one structure requires singular and the other requires plural.

8.4 Feature-based rules

S → NP VP

NP → DET N

VP → V NP

VP → V PP

VP → V S

PP → P NP

Constraints:

- Subject NP agreement must match VP agreement.
- Determiner number must match noun number.
- Verb subcategorization must match the complement.
- Pronoun case must match its grammatical role.

For example, The student submits the form succeeds because both NP and VP are singular, third-person. The students submits the form fails unification.

8.5 Integration with Earley

Feature constraints can be checked during prediction and completion. During COMPLETER, a predicted category is unified with the completed category. If unification fails, that parse is discarded.

This improves grammatical accuracy but adds unification operations. The practical cost depends on the number and complexity of features.

TASK 9 – Probabilistic Context-Free Grammars 5 Marks

Guiding Enquiry

When a query has several valid parses, how can probabilities select the most plausible one?

Investigate and interpret:

- Define a PCFG: each rule A → β has a probability P(A → β | A), with the probabilities for each left-hand side summing to 1. Show that your grammar satisfies this condition.
- Estimate rule probabilities by maximum likelihood from your 20-sentence mini treebank, giving the counts, the formula and the resulting table; explain how you handle unseen rules.
- Compute the probability of at least two parse trees for an ambiguous sentence (e.g., PP-attachment) and choose the best parse; compute the probability of the sentence as the sum over its trees.

- Describe the probabilistic CKY (Viterbi) algorithm in pseudocode after conversion to CNF, and compute the table for a short sentence.
- Discuss the independence assumptions of PCFGs (context-freeness, position invariance), the preference for shorter trees, and the lack of lexical sensitivity, illustrating each with a failure case.

Evidence Expected

A well-formed PCFG with estimated probabilities, worked tree probability calculations, a Viterbi trace and a reasoned discussion of limitations.

9.1 Definition

A Probabilistic Context-Free Grammar (PCFG) assigns a probability to each grammar rule:

\[P(A \rightarrow \beta \mid A) \]

For each left-hand-side non-terminal \(A\), its rule probabilities must sum to 1:

\[\sum_{\beta} P(A \rightarrow \beta \mid A)=1 \]

9.2 Maximum-likelihood estimation

Rule probabilities can be estimated from a treebank of annotated sentences:

\[P(A \rightarrow \beta)= \frac{\text{Count}(A\rightarrow\beta)} {\text{Total count of rules with left side }A} \]

Example illustrative counts:

Rule	Count	Probability
S → NP VP	18	18/20 = 0.90
S → VP	2	2/20 = 0.10

These are example counts only. The actual probabilities must be estimated from the assignment's annotated 20-sentence treebank.

For unseen rules, smoothing can assign a small probability rather than zero.

9.3 Parse-tree probability

The probability of a parse tree is the product of its rule probabilities.

For an ambiguous sentence:

Book the room near the library for the seminar.

Two possible parses:

- PP near the library attaches to NP room.
- PP near the library attaches to VP book.

The probability of each tree is calculated by multiplying the probabilities of all rules used in that tree.

The sentence probability is:

\[P(sentence)=\sum_{T \in Parses(sentence)}P(T) \]

The best parse is the tree with the highest probability.

9.4 Viterbi / probabilistic CKY

After converting the grammar to Chomsky Normal Form (CNF), CKY uses a chart to find the most probable parse.

for each word:

initialize chart with lexical categories

for span_length = 2 to n:

for each span:

for each split:

combine left and right categories

calculate rule probability

keep the highest-probability parse

The Viterbi score is:

\[V[A,i,j]=\max_{A\rightarrow BC,\ k} P(A\rightarrow BC)\times V[B,i,k]\times V[C,k,j] \]

9.5 PCFG limitations

- Context-free assumption: Rules do not directly account for broader context.
- Position invariance: A rule has the same probability wherever it occurs.
- Short-tree preference: Trees with fewer rule applications may receive higher probability.
- Limited lexical sensitivity: Basic PCFGs may not know that certain words strongly prefer particular attachments.

TASK 10 – Implement and Test the Parsers 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand? **Investigate and interpret:**

- Implement in Python (plain code or NLTK) the grammar, a top-down parser, an Earley parser and a Viterbi/PCFG parser, with feature checks for agreement and subcategorization.
- Run all parsers on the 30-sentence test set and report, for each: parse found or not, number of parses, number of chart states/steps and run-time. • Include test cases for left recursion, ambiguous attachment, ungrammatical sentences and long sentences (up to 20 words), and a reference check against your hand-computed results from Tasks 7 and 9.
- Present the results in comparable tables and at least one plot (e.g., number of states against sentence length).

Evidence Expected

Working, documented code, a reproducible test procedure and results tables/graphs.

10.1 Python tools

Use NLTK to implement:

- CFG grammar
- Recursive-descent parser
- Earley parser
- PCFG and Viterbi parser

A basic CFG example:

import nltkgrammar = nltk.CFG.fromstring("""S -> NP VPNP -> Det NVP -> V NPDet -> 'the'N -> 'student' | 'fee'V -> 'pays'""")parser = nltk.RecursiveDescentParser(grammar)sentence = "the student pays the fee".split()for tree in parser.parse(sentence): print(tree)

10.2 Testing procedure

Run all parsers on the 30-sentence test set, including:

- Declarative sentences
- Imperatives
- Yes/no questions

- Wh-questions
- PP-attachment ambiguity
- Ungrammatical agreement examples
- Sentences up to 20 words

For every sentence, record whether a parse is found, the number of parses, the number of states or steps, and runtime.

10.3 Results table

Parser	Parse found?	Number of parses	States/steps	Runtime
Top-down	To test	To measure	To measure	To measure
Earley	To test	To measure	To measure	To measure
PCFG/Viterbi	To test	Best parse	To measure	To measure

TASK 11 – Compare, Recommend and Defend the Parsing Design 2 Marks

Guiding Enquiry

Which grammar-and-parser combination would you deploy in the helpdesk, and what evidence supports your choice?

Investigate and interpret:

- Compare the top-down, Earley and probabilistic parsers on coverage, ambiguity handling, cost and ease of grammar maintenance, using your analysis and measured results.
- Recommend a design (e.g., feature-based CFG + Earley + PCFG ranking), stating its expected accuracy and limitations.
- Discuss manual effort (grammar writing and annotation) and the point at which a statistical approach becomes more economical than hand-written rules.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the fastest option alone is not sufficient.

11.1 Comparison

Parser	Strength	Limitation

Top-down	Simple and goal-driven	Can backtrack heavily; left recursion problem
Earley	Handles left recursion and ambiguity through a chart	More complex than recursive descent
PCFG/Viterbi	Selects the most probable parse	Needs annotated data and may rank incorrectly

11.2 Recommendation

I recommend a feature-based CFG with Earley parsing and PCFG ranking.

- Feature structures enforce agreement and subcategorization.
- Earley parsing handles ambiguity and left recursion.
- PCFG ranking selects the most plausible parse among valid alternatives.

The system should be evaluated using parse coverage, parse accuracy, runtime, and chart size. Exact accuracy figures should be reported only after testing the 30-sentence set.

Manual grammar writing and treebank annotation require time. A statistical approach becomes more economical when the system needs broad coverage and enough annotated data is available.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The helpdesk must now accept conversational, multi-clause and partly ungrammatical student messages (for example, “Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?”). Messages may contain coordinated clauses, embedded questions, relative clauses and missing punctuation. At least 20% of the incoming messages are expected to contain a construction not covered by your grammar.

Re-evaluate your original design under the changed conditions. You may: **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the constructions in the new messages that your grammar cannot parse (embedded questions, multi-clause coordination, ellipsis, fragments).
- Identify the affected assumptions: controlled vocabulary, fully grammatical input, small ambiguity, hand-written coverage.
- Estimate how the longer sentences affect the number of parses and the parsing time.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended parser behave on the new input?

- Run or hand-trace at least three new messages through your original grammar and parsers and record what fails first.
- Estimate the growth of ambiguity and chart size with sentence length using your Task 7 and Task 10 results.
- Identify whether another parsing strategy handles the new input more economically.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the grammar and parser be extended, replaced, or supplemented?

- Choose retain, modify or redesign (e.g., add rules for embedded clauses, a robust/partial-parsing fallback, feature-based gap threading, a larger PCFG estimated from more data).
- Describe the revised grammar and parser and what changes in rules, features and probabilities.
- Determine the effect on coverage, ambiguity, parsing cost and annotation effort.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one given the evidence you generated?

- Use your derivations, traces, simulation results, coverage figures and effort estimates.
- Show that the decision follows logically from the evidence and state its remaining limitations.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

A. Impact Analysis (2 Marks)

The helpdesk must now handle conversational and multi-clause messages such as:

Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?

The original controlled CFG may not handle:

- Embedded questions: tell me when the exams will be held
- Relative clauses: the exams that were postponed
- Coordinated clauses: and also whether the hostel fee is due
- Fragments: Hostel fee due?
- Missing punctuation and informal wording

The original assumptions of grammatical input, limited ambiguity, and a controlled vocabulary no longer hold.

Longer sentences create more possible attachment and clause structures. The number of possible parses may increase rapidly, while chart parsing stores more states. Exact growth should be measured using the parser.

B. Evaluation of the Original Design (2 Marks)

Test at least three new messages:

Message	**New construction**	**Expected challenge**
Can you tell me when the exams will be held?	Embedded wh-question	Original question rules may not support embedded clauses
The exams that were postponed	Relative clause	Relative clause rule and agreement

will be held tomorrow.		required
Is the hostel fee due, and can I pay it online?	Coordinated questions	Coordination of sentence-level constructions

For each message, record whether the original grammar parses it, the number of parses, and the number of parser states.

An Earley parser can handle left recursion and reuse chart states more economically than naïve top-down parsing. However, it cannot parse constructions that are absent from the grammar.

C. Adaptation or Redesign (2 Marks)

Decision: Modify the original system.

Retain the CFG, feature structures, Earley parser, and PCFG ranking, but extend them.

Add rules for embedded clauses:

VP → V NP CP

CP → COMP S

Add relative clauses:

NP → NP REL

REL → RELPRO VP

Add coordinated clauses:

S → S CONJ S

Add feature information to track agreement and long-distance dependencies. Introduce a partial-parsing fallback for fragments and ungrammatical messages.

Use a larger annotated corpus to estimate PCFG probabilities for the new constructions.

Expected effects:

- Coverage: Increases because more sentence forms are supported.
- Ambiguity: May increase because more structures can be generated.
- Parsing cost: May increase due to the larger grammar and longer messages.
- Annotation effort: Increases because new constructions need parsed examples.

The exact changes in coverage, ambiguity, and runtime must be measured.

D. Evidence-Based Defence (2 Marks)

The modified design is suitable because it preserves the original grammar-based structure while extending coverage for conversational helpdesk messages.

Earley parsing is preferred over simple top-down parsing because it avoids repeated work by storing intermediate states. Feature structures enforce agreement and subcategorization. PCFG ranking helps select a likely parse when multiple valid structures exist.

The final decision should be supported by:

- Results on the original 30-sentence test set
- Results on the three new messages
- Parse coverage and number of parses
- Chart-state counts and runtime

MARK DISTRIBUTION

Task	Marks
Task 1 – Design a Context-Free Grammar for English Syntax	5
Task 2 – Context-Free Rules, Derivations and Parse Trees	3
Task 3 – Sentence-Level Constructions	4
Task 4 – Agreement	3
Task 5 – Subcategorization	4
Task 6 – Top-Down Parsing	4
Task 7 – Earley Parsing	5
Task 8 – Feature Structures and Unification	4

Task 9 – Probabilistic Context-Free Grammars	5
Task 10 – Implement and Test the Parsers	3
Task 11 – Compare, Recommend and Defend the Parsing Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**